

Resenha - Software Architecture A Roadmap - David Garlan

Neste artigo, David Garlan examina a evolução da arquitetura de software, saindo de uma prática ad hoc para se tornar uma disciplina essencial da engenharia de software. Garlan argumenta que a escolha correta da arquitetura é um fator crítico de sucesso, afetando diretamente o desempenho, escalabilidade e manutenibilidade de sistemas complexos.

A Arquitetura como ponte

A proposta central de Garlan é que a arquitetura atua como uma ponte entre os requisitos e a implementação.

Abstração e Visibilidade: A arquitetura fornece uma descrição abstrata que expõe propriedades críticas do sistema enquanto esconde detalhes de implementação irrelevantes para o design de alto nível.

Tomada de Decisão: Permite que designers realizem análises críticas e avaliem se o sistema satisfará os requisitos antes da construção completa.

Pontos de Análise

Garlan identifica seis funções fundamentais onde a arquitetura agrega valor ao desenvolvimento:

Compreensão: Simplifica sistemas grandes ao apresentá-los em um nível de abstração tratável intelectualmente.

Reuso: Suporta o reuso não apenas de componentes, mas de estruturas completas e padrões de design.

Construção: Atua como um esboço (blueprint) que define componentes principais e suas dependências.

Evolução: Expõe as "paredes mestras" do sistema, permitindo prever custos de mudanças e separar a funcionalidade dos mecanismos de interação.

Análise: Permite checagens de consistência e conformidade com atributos de qualidade.

Gerenciamento: Serve como um marco (milestone) crítico para avaliação de riscos e estratégias de implementação.

Desafios e Tendências

Garlan aponta que o campo ainda está amadurecendo e enfrenta novos desafios tecnológicos:

Build-versus-Buy: A pressão econômica exige o uso massivo de código externo, transformando desenvolvedores em "integradores de sistemas" que precisam de padrões arquiteturais para evitar incompatibilidades.

Computação Centrada na Rede: A transição para sistemas abertos como a Internet exige arquiteturas que suportem coalizões dinâmicas de recursos autônomos.

Computação Pervasiva: A explosão de dispositivos heterogêneos requer arquiteturas flexíveis que lidem com recursos limitados e reconfiguração dinâmica.

Exemplos Práticos

Para um desenvolvimento de um sistema de rastreamento de veículos em tempo real que deve ser escalável e suportar diferentes tipos de sensores.

1. A Arquitetura como Ponte (Requisitos -> Código)

Requisito: O sistema deve processar dados de 10.000 veículos simultaneamente com baixa latência.

Ponto de Garlan: Em vez de codificar diretamente, define-se um estilo arquitetural de "Filtros e Tubos" (Pipe-and-Filter). Isso permite analisar gargalos e capacidades de buffer antes de escrever uma única linha de lógica de processamento.

2. Evolução e Manutenibilidade

Cenário: O fornecedor dos rastreadores GPS é alterado, mudando o formato dos dados de entrada.

Aplicação: Seguindo a separação de preocupações defendida por Garlan, a mudança afeta apenas o componente de "Ingestão", mantendo os módulos de "Análise" e "Visualização" intactos, pois a interface de interação (conector) foi bem definida na arquitetura.

3. Integração de Terceiros (Build-versus-Buy)

Cenário: O sistema precisa integrar um serviço externo de mapas (Google Maps) e um serviço de faturamento.

Aplicação: O arquiteto utiliza padrões de integração para "consertar" possíveis incompatibilidades (mismatches) entre o que o serviço externo fornece e o que o sistema interno espera, tratando a integração como uma atividade de design explícita.